

homework 8, version 3

Homework 8: Epidemic modeling II

18.S191, Fall 2023

This notebook contains *built-in, live answer checks*! In some exercises you will see a coloured box, which runs a test case on your code, and provides feedback based on the result. Simply edit the code, run it, and the check runs again.

Feel free to ask questions!

Initializing packages

When running this notebook for the first time, this could take up to 15 minutes. Hang in there!

```
1 using Plots, PlutoUI
```

In the past couple homeworks, we've messed around with probability distributions and Monte Carlo simulations. This time, we'll build a more mathematical model, and we'll use optimization to fit it to given data.

Models have *parameters*, such as the rate of recovery from infection. Where do the parameter values come from? Ideally we would like to extract them from data. The goal of this homework is to do this by *fitting* a model to data.

For simplicity, we will use data that generated from a *spatial* model - one that simulates the movement of a population in space - instead of real-world data, and we will fit the simplest SIR model. But the same ideas apply more generally.

There are many ways to fit a function to data, but all must involve some form of **optimization**, usually **minimization** of a particular function, a **loss function**; this is the basis of the vast field of **machine learning**.

The loss function is a function of the model parameters; it measures *how far* the model *output* is from the data, for the given values of the parameters.

We emphasise that this material is pedagogical; there is no suggestion that these specific techniques should be used actual calculations; rather, it is the underlying ideas that are important.

Exercise 1: Calculus without calculus

Before we jump in to simulating the SIR equations, let's experiment with a simple 1D function. In calculus, we learn techniques for differentiating and integrating *symbolic* equations, e.g.

$\frac{d}{dx} x^n = nx^{n-1}$. But in real applications, it is often impossible to apply these techniques, either because the problem is too complicated to solve symbolically, or because our problem has no symbolic expression, like when working with experimental results.

Instead, as you will recall from lecture, we use 🦄 *computers* 🦄 to approximate derivatives and integrals. Instead of applying rules to symbolic expressions, we use much simpler strategies that *only use the output values of our function*.

One such example is the *finite difference* method for approximating the *derivative* of a function. It is inspired by the analytical definition of the derivative:

$$f'(a) := \lim_{h \rightarrow 0} \frac{f(a+h) - f(a)}{h}.$$

The finite difference method simply fixes a small value for h , say $h = 10^{-3}$, and then approximates the derivative as:

$$f'(a) \simeq \frac{f(a+h) - f(a)}{h}.$$

Exercise 1.1 - *tangent line*

👉 Write a function `finite_difference_slope` that takes a function `f` and numbers `a` and `h`. It returns the slope $f'(a)$, approximated using the finite difference formula above.

`finite_difference_slope` (generic function with 2 methods)

```
1 function finite_difference_slope(f::Function, a, h=1e-3)
2
3     return (f(a+h)-f(a))/h
4 end
```

0.2

```
1 finite_difference_slope(sqrt, 4.0, 5.0)
```

Got it!

Good job!

👉 Write a function `tangent_line` that takes the same arguments `f`, `a` and `g`, but it **returns a function**. This function ($\mathbb{R} \rightarrow \mathbb{R}$) is the *tangent line* with slope $f'(a)$ (computed using `finite_difference_slope`) that passes through $(a, f(a))$.

Hint

Remember that functions are objects! For example, here is a function that returns the square root function:

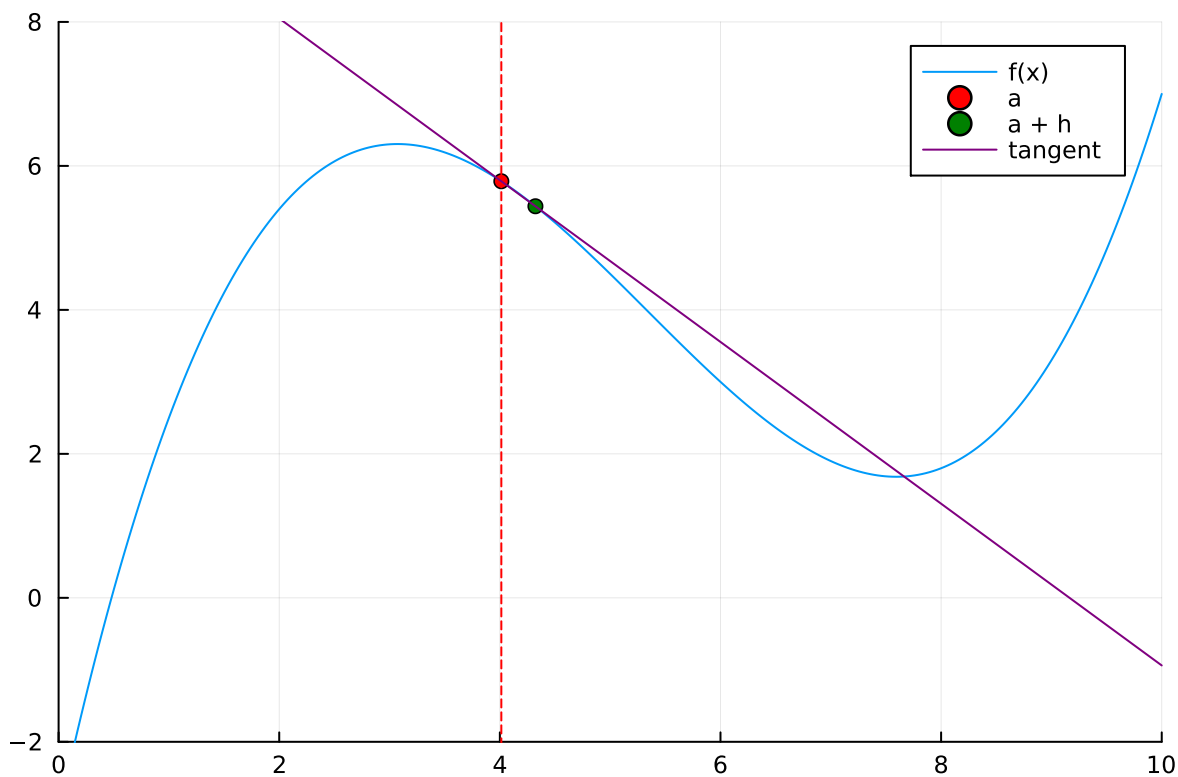
```
function the_square_root_function()
    f(x) = sqrt(x)
    return f
end
```

tangent_line (generic function with 1 method)

```
1 function tangent_line(f, a, h)
2     m=finite_difference_slope(f,a,h)
3     c=f(a)-m*a
4     g=p -> p*m+c
5
6     return g
7 end
```

Got it!

Great! 🎉



```
1 # this is our test function
2 wavy(x) = .1x^3 - 1.6x^2 + 7x - 3;
```

The slider below controls h using a *log scale*. In the (mathematical) definition of the derivative, we take $\lim_{h \rightarrow 0}$. This corresponds to moving the slider to the left.

Notice that, as you decrease h , the tangent line gets more accurate, but what happens if you make h too small?



`h_finite_diff = 0.30902954325135906`

Exercise 1.2 - antiderivative

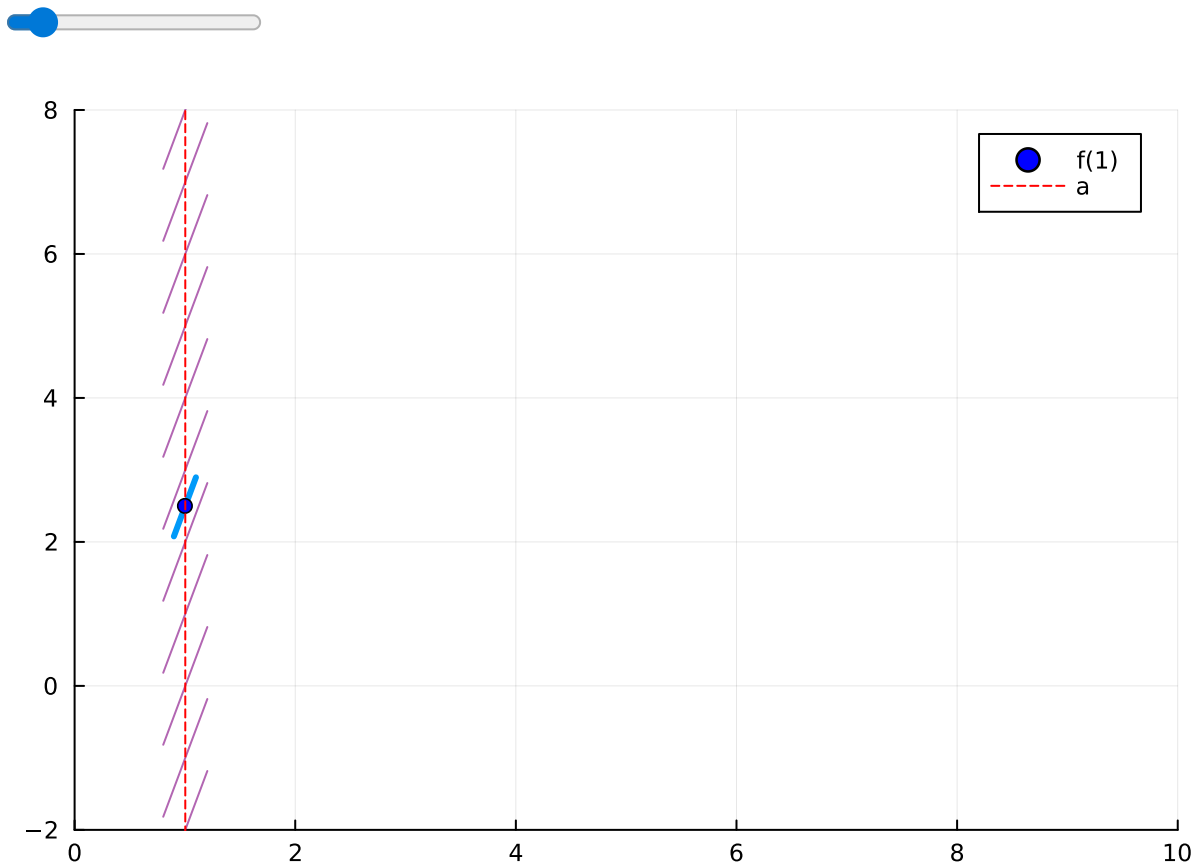
In the finite differences method, we approximated the derivative of a function:

$$f'(a) \simeq \frac{f(a+h) - f(a)}{h}$$

We can do something very similar to approximate the 'antiderivate' of a function. Finding the antiderivative means that we use the *slope* f' to compute f numerically!

This antiderivative problem is illustrated below. The only information that we have is the **slope** at any point $a \in \mathbb{R}$, and we have one **initial value**, $f(1)$.

```
1 # in this exercise, only the derivative is given
2 wavy_deriv(x) = .3x^2 - 3.2x + 7;
```



Using only this information, we want to **reconstruct** f .

By rearranging the equation above, we get the *Euler method*:

$$f(a+h) \simeq hf'(a) + f(a)$$

Using this formula, we only need to know the *value* $f(a)$ and the *slope* $f'(a)$ of a function at a to get the value at $a+h$. Doing this repeatedly can give us the value at $a+2h$, at $a+3h$, etc., all from one initial value $f(a)$.

✍ Write a function `euler_integrate_step` that applies this formula to a known function f' at a , with step size h and the initial value $f(a)$. It returns the next value, $f(a + h)$.

`euler_integrate_step` (generic function with 1 method)

```
1 function euler_integrate_step(fprime::Function, fa::Number,  
2     a::Number, h::Number)  
3  
4     return fprime(a+h)*h+fa  
5 end
```

Got it!

Let's move on to the next section.

```
1 let  
2     result = euler_integrate_step(x -> x^2, 10, 11, 12)  
3  
4     if result isa Missing  
5         still_missing()  
6     elseif !(result isa Number)  
7         keep_working(md"Make sure that you return a number.")  
8     else  
9         if result ≈ 6358  
10             correct()  
11         elseif result ≈ 1462  
12             almost(md"Use ``f'(a+h)``, not ``f'(a)``.")  
13         else  
14             keep_working()  
15         end  
16     end  
17 end
```

✍ Write a function `euler_integrate` that takes takes a known function f' , the initial value $f(a)$ and a range T with $a == \text{first}(T)$ and $h == \text{step}(T)$. It applies the function `euler_integrate_step` repeatedly, once per entry in T , to produce the sequence of values $f(a + h)$, $f(a + 2h)$, etc.

`euler_integrate` (generic function with 1 method)

```
1 function euler_integrate(fprime::Function, fa::Number,  
2     T::AbstractRange)  
3  
4     a0 = T[1]  
5     h = step(T)  
6     ans=ones(length(T))  
7     ans[1]=fa  
8  
9     for i in 2:length(T)  
10         ans[i]=euler_integrate_step(fprime,ans[i-1],T[i-1],h)  
11     end  
12     return ans  
13 end
```

Let's try it out on $f'(x) = 3x^2$ and T ranging from 0 to 10.

We already know the analytical solution $f(x) = x^3$, so the result should be an array going from (approximately) 0.0 to 1000.0.

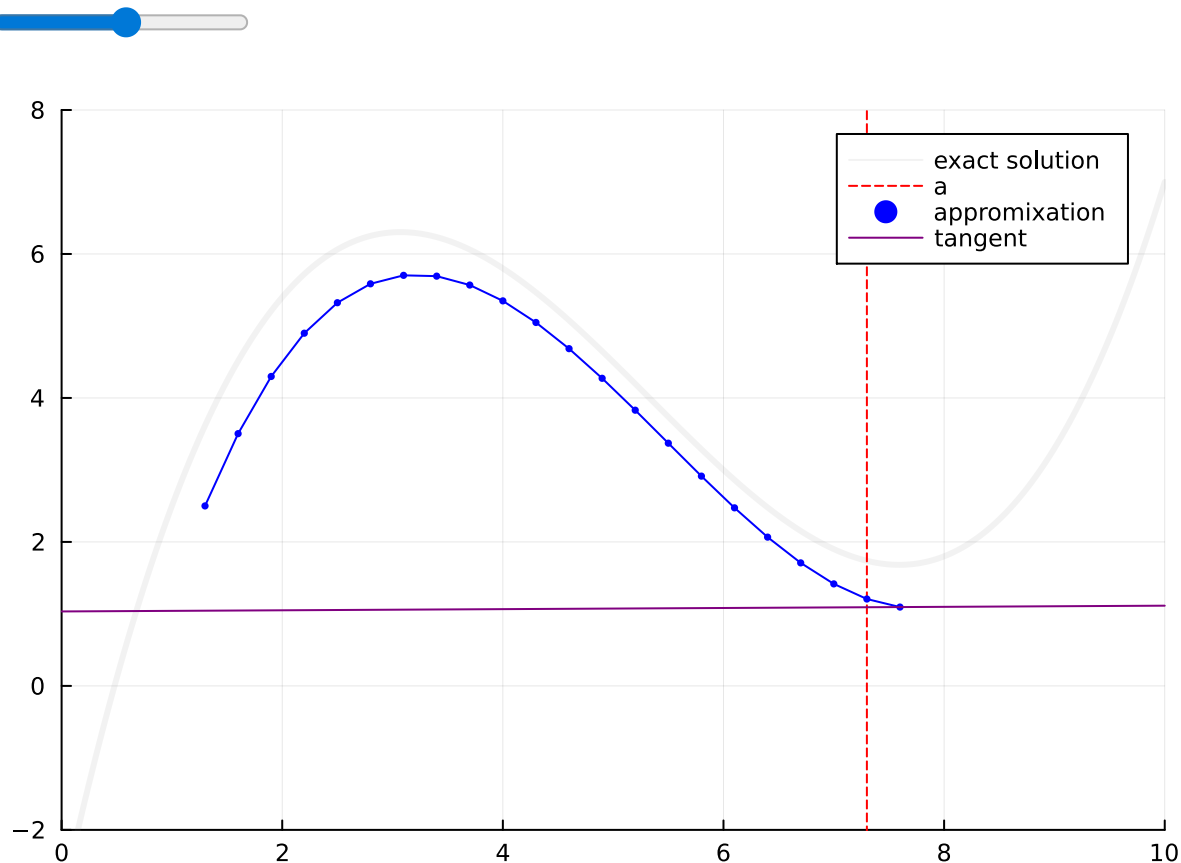
`euler_test =`

[0.0, 0.003, 0.015, 0.042, 0.09, 0.165, 0.273, 0.42, 0.612, 0.855, 1.155, 1.518, 1.95, 2.457,

```
1 euler_test = let
2   fprime(x) = 3x^2
3   T = 0 : 0.1 : 10
4
5   euler_integrate(fprime, 0, T)
6 end
```

Got it!

Keep it up!



You see that our numerical antiderivate is not very accurate, but we can get a smaller error by choosing a smaller step size. Try it out!

There are also alternative integration methods that are more accurate with the same step size. Some methods also use the second derivative, other methods use multiple steps at once, etc.! This is the study of Numerical Methods.

Exercise 2: Simulating the SIR differential equations

We will look at a type of mathematical model for epidemic outbreaks called the **SIR model**. In this model, we consider three functions of time t : the fraction of the population $s(t)$ that is *susceptible* to the disease, the fraction $i(t)$ that is *infected*, and the fraction $r(t)$ that has *recovered*. We've already used these three quantities in a simulation before, namely in the Monte Carlo simulation from last homework!

It turns out that, given some assumptions about how people from the three groups interact over time, we can model the behavior of the three quantities in terms of their **derivatives** with respect to time t . We use two parameters β and γ , and with them, we can form a system of ordinary differential equations (ODEs) for the SIR model. They are as follows:

$$\begin{aligned}\dot{s} &= -\beta s i \\ \dot{i} &= +\beta s i - \gamma i \\ \dot{r} &= +\gamma i\end{aligned}$$

where $\dot{s} := \frac{ds}{dt}$ is the derivative of s with respect to time. Recall that s denotes the *proportion* (fraction) of the population that is susceptible, a number between 0 and 1.

We will use the simplest possible method to simulate these, namely the **Euler method**. The Euler method is not always a good method to solve ODEs accurately, but for our purposes it is good enough.

In the previous exercise, we introduced the Euler method for a 1D function, which you can see as an ODE that only depends on time. For the SIR equations, we have an ODE that only depends on the previous *value*, not on time, and we have 3 equations instead of 1.

The solution is quite simple, we apply the Euler method to *each* of the differential equations within a *single* time step to get new values for *each* of s , i and r at the end of the time step in terms of the values at the start of the time step. The Euler discretised equations are:

$$\begin{aligned}s(t+h) &= s(t) - h \cdot \beta s(t) i(t) \\ i(t+h) &= i(t) + h \cdot (\beta s(t) i(t) - \gamma i(t)) \\ r(t+h) &= r(t) + h \cdot \gamma i(t)\end{aligned}$$

👉 Implement a function `euler_SIR_step( $\beta$ ,  $\gamma$ , sir_0,  $h$ )` that performs a single Euler step for these equations with the given parameter values and initial values, with a step size h .

`sir_0` is a 3-element vector, and you should return a new 3-element vector with the values after the timestep.

euler_SIR_step (generic function with 1 method)

```
1 function euler_SIR_step( $\beta$ ,  $\gamma$ , sir_0::Vector, h::Number)
2     s, i, r = sir_0
3
4     return [
5         s-h* $\beta$ *s*i,
6         i+h*( $\beta$ *s*i- $\gamma$ *i),
7         r+h* $\gamma$ *i,
8     ]
9 end
```

```
[0.989901, 0.010049, 5.0e-5]
```

```
1 euler_SIR_step(0.1, 0.05,
2     [0.99, 0.01, 0.00],
3     0.1)
```

👉 Implement a function `euler_SIR( $\beta$ ,  $\gamma$ , sir_0, T)` that applies the previously defined function over a time range T .

You should return a vector of vectors: a 3-element vector for each point in time.

euler_SIR (generic function with 1 method)

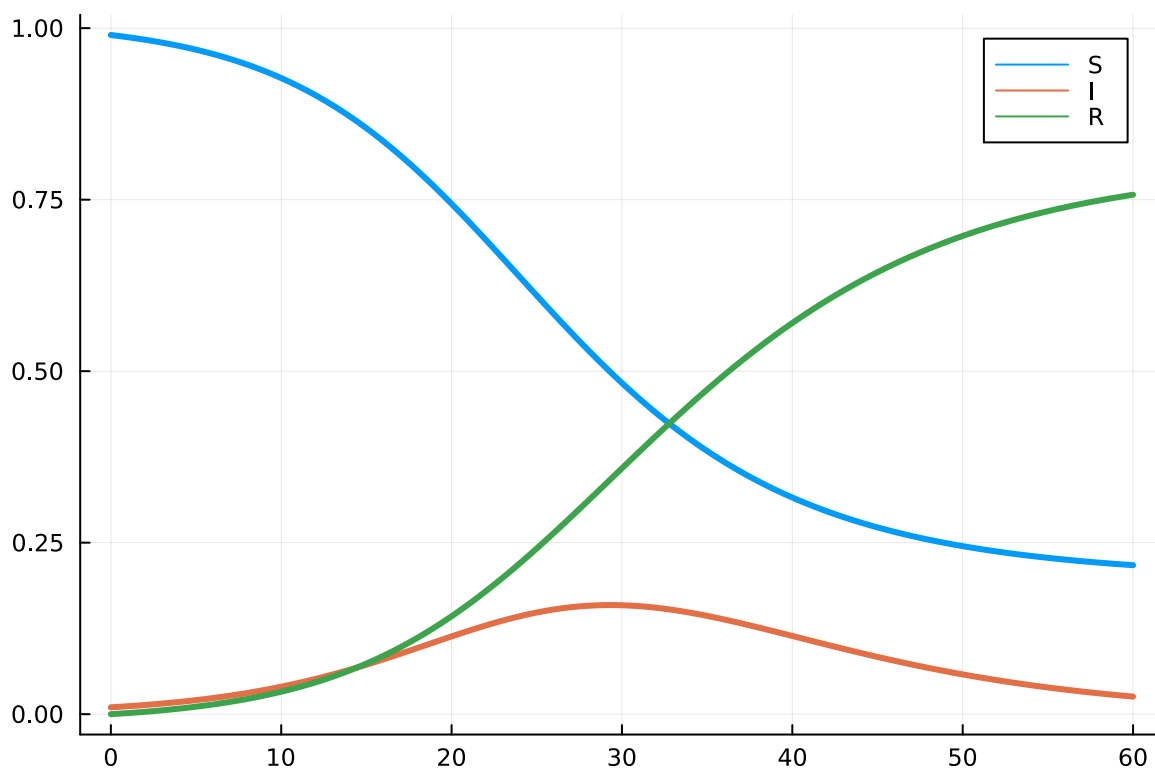
```
1 function euler_SIR( $\beta$ ,  $\gamma$ , sir_0::Vector, T::AbstractRange)
2     # T is a range, you get the step size and number of steps like so:
3     h = step(T)
4
5     num_steps = length(T)
6     ans=Vector{Vector{Float64}}(undef,length(T))
7     ans[1]=sir_0
8     for i in 2:length(T)
9         ans[i]=euler_SIR_step( $\beta$ , $\gamma$ ,ans[i-1],h)
10    end
11
12    return ans
13 end
```

```
sir_T = 0.0:0.1:60.0
```

```
1 sir_T = 0 : 0.1 : 60.0
```

```
sir_results =
```

```
[[0.99, 0.01, 0.0], [0.989703, 0.010147, 0.00015], [0.989402, 0.0102961, 0.000302205], [0.989101, 0.0104421, 0.000450395], [0.988801, 0.0105871, 0.000598595], [0.988501, 0.0107311, 0.000746795], [0.988201, 0.0108751, 0.000894995], [0.987901, 0.0110191, 0.001043195], [0.987601, 0.0111631, 0.001191395], [0.987301, 0.0113071, 0.001339595], [0.987, 0.0114511, 0.001487795], [0.9867, 0.0115951, 0.001635995], [0.9864, 0.0117391, 0.001784195], [0.9861, 0.0118831, 0.001932395], [0.9858, 0.0120271, 0.002080595], [0.9855, 0.0121711, 0.002228795], [0.9852, 0.0123151, 0.002376995], [0.9849, 0.0124591, 0.002525195], [0.9846, 0.0126031, 0.002673395], [0.9843, 0.0127471, 0.002821595], [0.984, 0.0128911, 0.002969795], [0.9837, 0.0130351, 0.003117995], [0.9834, 0.0131791, 0.003266195], [0.9831, 0.0133231, 0.003414395], [0.9828, 0.0134671, 0.003562595], [0.9825, 0.0136111, 0.003710795], [0.9822, 0.0137551, 0.003858995], [0.9819, 0.0138991, 0.004007195], [0.9816, 0.0140431, 0.004155395], [0.9813, 0.0141871, 0.004303595], [0.981, 0.0143311, 0.004451795], [0.9807, 0.0144751, 0.004599995], [0.9804, 0.0146191, 0.004748195], [0.9801, 0.0147631, 0.004896395], [0.9798, 0.0149071, 0.005044595], [0.9795, 0.0150511, 0.005192795], [0.9792, 0.0151951, 0.005340995], [0.9789, 0.0153391, 0.005489195], [0.9786, 0.0154831, 0.005637395], [0.9783, 0.0156271, 0.005785595], [0.978, 0.0157711, 0.005933795], [0.9777, 0.0159151, 0.006081995], [0.9774, 0.0160591, 0.006230195], [0.9771, 0.0162031, 0.006378395], [0.9768, 0.0163471, 0.006526595], [0.9765, 0.0164911, 0.006674795], [0.9762, 0.0166351, 0.006822995], [0.9759, 0.0167791, 0.006971195], [0.9756, 0.0169231, 0.007119395], [0.9753, 0.0170671, 0.007267595], [0.975, 0.0172111, 0.007415795], [0.9747, 0.0173551, 0.007563995], [0.9744, 0.0174991, 0.007712195], [0.9741, 0.0176431, 0.007860395], [0.9738, 0.0177871, 0.008008595], [0.9735, 0.0179311, 0.008156795], [0.9732, 0.0180751, 0.008304995], [0.9729, 0.0182191, 0.008453195], [0.9726, 0.0183631, 0.008601395], [0.9723, 0.0185071, 0.008749595], [0.972, 0.0186511, 0.008897795], [0.9717, 0.0187951, 0.009045995], [0.9714, 0.0189391, 0.009194195], [0.9711, 0.0190831, 0.009342395], [0.9708, 0.0192271, 0.009490595], [0.9705, 0.0193711, 0.009638795], [0.9702, 0.0195151, 0.009786995], [0.9699, 0.0196591, 0.009935195], [0.9696, 0.0198031, 0.010083395], [0.9693, 0.0199471, 0.010231595], [0.969, 0.0200911, 0.010379795], [0.9687, 0.0202351, 0.010527995], [0.9684, 0.0203791, 0.010676195], [0.9681, 0.0205231, 0.010824395], [0.9678, 0.0206671, 0.010972595], [0.9675, 0.0208111, 0.011120795], [0.9672, 0.0209551, 0.011268995], [0.9669, 0.0210991, 0.011417195], [0.9666, 0.0212431, 0.011565395], [0.9663, 0.0213871, 0.011713595], [0.966, 0.0215311, 0.011861795], [0.9657, 0.0216751, 0.012009995], [0.9654, 0.0218191, 0.012158195], [0.9651, 0.0219631, 0.012306395], [0.9648, 0.0221071, 0.012454595], [0.9645, 0.0222511, 0.012602795], [0.9642, 0.0223951, 0.012750995], [0.9639, 0.0225391, 0.012899195], [0.9636, 0.0226831, 0.013047395], [0.9633, 0.0228271, 0.013195595], [0.963, 0.0229711, 0.013343795], [0.9627, 0.0231151, 0.013491995], [0.9624, 0.0232591, 0.013640195], [0.9621, 0.0234031, 0.013788395], [0.9618, 0.0235471, 0.013936595], [0.9615, 0.0236911, 0.014084795], [0.9612, 0.0238351, 0.014232995], [0.9609, 0.0239791, 0.014381195], [0.9606, 0.0241231, 0.014529395], [0.9603, 0.0242671, 0.014677595], [0.96, 0.0244111, 0.014825795], [0.9597, 0.0245551, 0.014973995], [0.9594, 0.0246991, 0.015122195], [0.9591, 0.0248431, 0.015270395], [0.9588, 0.0249871, 0.015418595], [0.9585, 0.0251311, 0.015566795], [0.9582, 0.0252751, 0.015714995], [0.9579, 0.0254191, 0.015863195], [0.9576, 0.0255631, 0.016011395], [0.9573, 0.0257071, 0.016159595], [0.957, 0.0258511, 0.016307795], [0.9567, 0.0259951, 0.016455995], [0.9564, 0.0261391, 0.016604195], [0.9561, 0.0262831, 0.016752395], [0.9558, 0.0264271, 0.016900595], [0.9555, 0.0265711, 0.017048795], [0.9552, 0.0267151, 0.017196995], [0.9549, 0.0268591, 0.017345195], [0.9546, 0.0270031, 0.017493395], [0.9543, 0.0271471, 0.017641595], [0.954, 0.0272911, 0.017789795], [0.9537, 0.0274351, 0.017937995], [0.9534, 0.0275791, 0.018086195], [0.9531, 0.0277231, 0.018234395], [0.9528, 0.0278671, 0.018382595], [0.9525, 0.0280111, 0.018530795], [0.9522, 0.0281551, 0.018678995], [0.9519, 0.0282991, 0.018827195], [0.9516, 0.0284431, 0.018975395], [0.9513, 0.0285871, 0.019123595], [0.951, 0.0287311, 0.019271795], [0.9507, 0.0288751, 0.019419995], [0.9504, 0.0290191, 0.019568195], [0.9501, 0.0291631, 0.019716395], [0.9498, 0.0293071, 0.019864595], [0.9495, 0.0294511, 0.020012795], [0.9492, 0.0295951, 0.020160995], [0.9489, 0.0297391, 0.020309195], [0.9486, 0.0298831, 0.020457395], [0.9483, 0.0300271, 0.020605595], [0.948, 0.0301711, 0.020753795], [0.9477, 0.0303151, 0.020901995], [0.9474, 0.0304591, 0.021050195], [0.9471, 0.0306031, 0.021198395], [0.9468, 0.0307471, 0.021346595], [0.9465, 0.0308911, 0.021494795], [0.9462, 0.0310351, 0.021642995], [0.9459, 0.0311791, 0.021791195], [0.9456, 0.0313231, 0.021939395], [0.9453, 0.0314671, 0.022087595], [0.945, 0.0316111, 0.022235795], [0.9447, 0.0317551, 0.022383995], [0.9444, 0.0318991, 0.022532195], [0.9441, 0.0320431, 0.022680395], [0.9438, 0.0321871, 0.022828595], [0.9435, 0.0323311, 0.022976795], [0.9432, 0.0324751, 0.023124995], [0.9429, 0.0326191, 0.023273195], [0.9426, 0.0327631, 0.023421395], [0.9423, 0.0329071, 0.023569595], [0.942, 0.0330511, 0.023717795], [0.9417, 0.0331951, 0.023865995], [0.9414, 0.0333391, 0.024014195], [0.9411, 0.0334831, 0.024162395], [0.9408, 0.0336271, 0.024310595], [0.9405, 0.0337711, 0.024458795], [0.9402, 0.0339151, 0.024606995], [0.9399, 0.0340591, 0.024755195], [0.9396, 0.0342031, 0.024903395], [0.9393, 0.0343471, 0.025051595], [0.939, 0.0344911, 0.025199795], [0.9387, 0.0346351, 0.025347995], [0.9384, 0.0347791, 0.025496195], [0.9381, 0.0349231, 0.025644395], [0.9378, 0.0350671, 0.025792595], [0.9375, 0.0352111, 0.025940795], [0.9372, 0.0353551, 0.026088995], [0.9369, 0.0354991, 0.026237195], [0.9366, 0.0356431, 0.026385395], [0.9363, 0.0357871, 0.026533595], [0.936, 0.0359311, 0.026681795], [0.9357, 0.0360751, 0.026829995], [0.9354, 0.0362191, 0.026978195], [0.9351, 0.0363631, 0.027126395], [0.9348, 0.0365071, 0.027274595], [0.9345, 0.0366511, 0.027422795], [0.9342, 0.0367951, 0.027570995], [0.9339, 0.0369391, 0.027719195], [0.9336, 0.0370831, 0.027867395], [0.9333, 0.0372271, 0.028015595], [0.933, 0.0373711, 0.028163795], [0.9327, 0.0375151, 0.028311995], [0.9324, 0.0376591, 0.028460195], [0.9321, 0.0378031, 0.028608395], [0.9318, 0.0379471, 0.028756595], [0.9315, 0.0380911, 0.028904795], [0.9312, 0.0382351, 0.029052995], [0.9309, 0.0383791, 0.029201195], [0.9306, 0.0385231, 0.029349395], [0.9303, 0.0386671, 0.029497595], [0.93, 0.0388111, 0.029645795], [0.9297, 0.0389551, 0.029793995], [0.9294, 0.0390991, 0.029942195], [0.9291, 0.0392431, 0.030090395], [0.9288, 0.0393871, 0.030238595], [0.9285, 0.0395311, 0.030386795], [0.9282, 0.0396751, 0.030534995], [0.9279, 0.0398191, 0.030683195], [0.9276, 0.0399631, 0.030831395], [0.9273, 0.0401071, 0.030979595], [0.927, 0.0402511, 0.031127795], [0.9267, 0.0403951, 0.031275995], [0.9264, 0.0405391, 0.031424195], [0.9261, 0.0406831, 0.031572395], [0.9258, 0.0408271, 0.031720595], [0.9255, 0.0409711, 0.031868795], [0.9252, 0.0411151, 0.032016995], [0.9249, 0.0412591, 0.032165195], [0.9246, 0.0414031, 0.032313395], [0.9243, 0.0415471, 0.032461595], [0.924, 0.0416911, 0.032609795], [0.9237, 0.0418351, 0.032757995], [0.9234, 0.0419791, 0.032906195], [0.9231, 0.0421231, 0.033054395], [0.9228, 0.0422671, 0.033202595], [0.9225, 0.0424111, 0.033350795], [0.9222, 0.0425551, 0.033498995], [0.9219, 0.0426991, 0.033647195], [0.9216, 0.0428431, 0.033795395], [0.9213, 0.0429871, 0.033943595], [0.921, 0.0431311, 0.034091795], [0.9207, 0.0432751, 0.034239995], [0.9204, 0.0434191, 0.034388195], [0.9201, 0.0435631, 0.034536395], [0.9198, 0.0437071, 0.034684595], [0.9195, 0.0438511, 0.034832795], [0.9192, 0.0439951, 0.034980995], [0.9189, 0.0441391, 0.035129195], [0.9186, 0.0442831, 0.035277395], [0.9183, 0.0444271, 0.035425595], [0.918, 0.0445711, 0.035573795], [0.9177, 0.0447151, 0.035721995], [0.9174, 0.0448591, 0.035870195], [0.9171, 0.0450031, 0.036018395], [0.9168, 0.0451471, 0.036166595], [0.9165, 0.0452911, 0.036314795], [0.9162, 0.0454351, 0.036462995], [0.9159, 0.0455791, 0.036611195], [0.9156, 0.0457231, 0.036759395], [0.9153, 0.0458671, 0.036907595], [0.915, 0.0460111, 0.037055795], [0.9147, 0.0461551, 0.037203995], [0.9144, 0.0462991, 0.037352195], [0.9141, 0.0464431, 0.037500395], [0.9138, 0.0465871, 0.037648595], [0.9135, 0.0467311, 0.037796795], [0.9132, 0.0468751, 0.037944995], [0.9129, 0.0470191, 0.038093195], [0.9126, 0.0471631, 0.038241395], [0.9123, 0.0473071, 0.038389595], [0.912, 0.0474511, 0.038537795], [0.9117, 0.0475951, 0.038685995], [0.9114, 0.0477391, 0.038834195], [0.9111, 0.0478831, 0.038982395], [0.9108, 0.0480271, 0.039130595], [0.9105, 0.0481711, 0.039278795], [0.9102, 0.0483151, 0.039426995], [0.9099, 0.0484591, 0.039575195], [0.9096, 0.0486031, 0.039723395], [0.9093, 0.0487471, 0.039871595], [0.909, 0.0488911, 0.040019795], [0.9087, 0.0490351, 0.040167995], [0.9084, 0.0491791, 0.040316195], [0.9081, 0.0493231, 0.040464395], [0.9078, 0.0494671, 0.040612595], [0.9075, 0.0496111, 0.040760795], [0.9072, 0.0497551, 0.040908995], [0.9069, 0.0498991, 0.041057195], [0.9066, 0.0500431, 0.041205395], [0.9063, 0.0501871, 0.041353595], [0.906, 0.0503311, 0.041501795], [0.9057, 0.0504751, 0.041649995], [0.9054, 0.0506191, 0.041798195], [0.9051, 0.0507631, 0.041946395], [0.9048, 0.0509071, 0.042094595], [0.9045, 0.0510511, 0.042242795], [0.9042, 0.0511951, 0.042390995], [0.9039, 0.0513391, 0.042539195], [0.9036, 0.0514831, 0.042687395], [0.9033, 0.0516271, 0.042835595], [0.903, 0.0517711, 0.042983795], [0.9027, 0.0519151, 0.043131995], [0.9024, 0.0520591, 0.043280195], [0.9021, 0.0522031, 0.043428395], [0.9018, 0.0523471, 0.043576595], [0.9015, 0.0524911, 0.043724795], [0.9012, 0.0526351, 0.043872995], [0.9009, 0.0527791, 0.044021195], [0.9006, 0.0529231, 0.044169395], [0.9003, 0.0530671, 0.044317595], [0.9, 0.0532111, 0.044465795], [0.8997, 0.0533551, 0.044613995], [0.8994, 0.0534991, 0.044762195], [0.8991, 0.0536431, 0.044910395], [0.8988, 0.0537871, 0.045058595], [0.8985, 0.0539311, 0.045206795], [0.8982, 0.0540751, 0.045354995], [0.8979, 0.0542191, 0.045503195], [0.8976, 0.0543631, 0.045651395], [0.8973, 0.0545071, 0.045799595], [0.897, 0.0546511, 0.045947795], [0.8967, 0.0547951, 0.046095995], [0.8964, 0.0549391, 0.046244195], [0.8961, 0.0550831, 0.046392395], [0.8958, 0.0552271, 0.046540595], [0.8955, 0.0553711, 0.046688795], [0.8952, 0.0555151, 0.046836995], [0.8949, 0.0556591, 0.046985195], [0.8946, 0.0558031, 0.047133395], [0.8943, 0.0559471, 0.047281595], [0.894, 0.0560911, 0.047429795], [0.8937, 0.0562351, 0.047577995], [0.8934, 0.0563791, 0.047726195], [0.8931, 0.0565231, 0.047874395], [0.8928, 0.0566671, 0.048022595], [0.8925, 0.0568111, 0.048170795], [0.8922, 0.0569551, 0.048318995], [0.8919, 0.0570991, 0.048467195], [0.8916, 0.0572431, 0.048615395], [0.8913, 0.0573871, 0.048763595], [0.891, 0.0575311, 0.048911795], [0.8907, 0.0576751, 0.049059995], [0.8904, 0.0578191, 0.049208195], [0.8901, 0.0579631, 0.049356395], [0.8898, 0.0581071, 0.049504595], [0.8895, 0.0582511, 0.049652795], [0.8892, 0.0583951, 0.049800995], [0.8889, 0.0585391, 0.049949195], [0.8886, 0.0586831, 0.050097395], [0.8883, 0.0588271, 0.050245595], [0.888, 0.0589711, 0.050393795], [0.8877, 0.0591151, 0.050541995], [0.8874, 0.0592591, 0.050690195], [0.8871, 0.0594031, 0.050838395], [0.8868, 0.0595471, 0.050986595], [0.8865, 0.0596911, 0.051134795], [0.8862, 0.0598351, 0.051282995], [0.8859, 0.0599791, 0.051431195], [0.8856, 0.0601231, 0.051579395], [0.8853, 0.0602671, 0.051727595], [0.885, 0.0604111, 0.051875795], [0.8847, 0.0605551, 0.052023995], [0.8844, 0.0606991, 0.052172195], [0.8841, 0.0608431, 0.052320395], [0.8838, 0.0609871, 0.052468595], [0.8835, 0.0611311, 0.052616795], [0.8832, 0.0612751, 0.052764995], [0.8829, 0.0614191, 0.052913195], [0.8826, 0.0615631, 0.053061395], [0.8823, 0.0617071, 0.053209595], [0.882, 0.0618511, 0.053357795], [0.8817, 0.0619951, 0.05350599
```



```
1 plot_sir!(plot(), sir_T, sir_results)
```

plot_sir! (generic function with 1 method)

```
1 function plot_sir!(p, T, results; label="", kwargs...)
2     s = getindex.(results, [1])
3     i = getindex.(results, [2])
4     r = getindex.(results, [3])
5
6     plot!(p, T, s; color=1, label=label*" S", lw=3, kwargs...)
7     plot!(p, T, i; color=2, label=label*" I", lw=3, kwargs...)
8     plot!(p, T, r; color=3, label=label*" R", lw=3, kwargs...)
9
10    p
11 end
```

👉 Do you see an epidemic outbreak (i.e. a rapid growth in number of infected individuals, followed by a decline)? What happens after a long time? Does everybody get infected?

```
1 md"""
2 👉 Do you see an epidemic outbreak (i.e. a rapid growth in number of infected
  individuals, followed by a decline)? What happens after a long time? Does everybody
  get infected?
3 """
```

default_SIR_parameters_observation =
your answer here

```
1 default_SIR_parameters_observation = md"""
2 _your answer here_
3 """
```

👉 Make an interactive visualization, similar to the above plot, in which you vary β and γ via sliders. What relation should β and γ have for an epidemic outbreak to occur?

0.35

```
1 @bind β Slider(0.0:0.01:1; show_value = true,default=0.3)
```

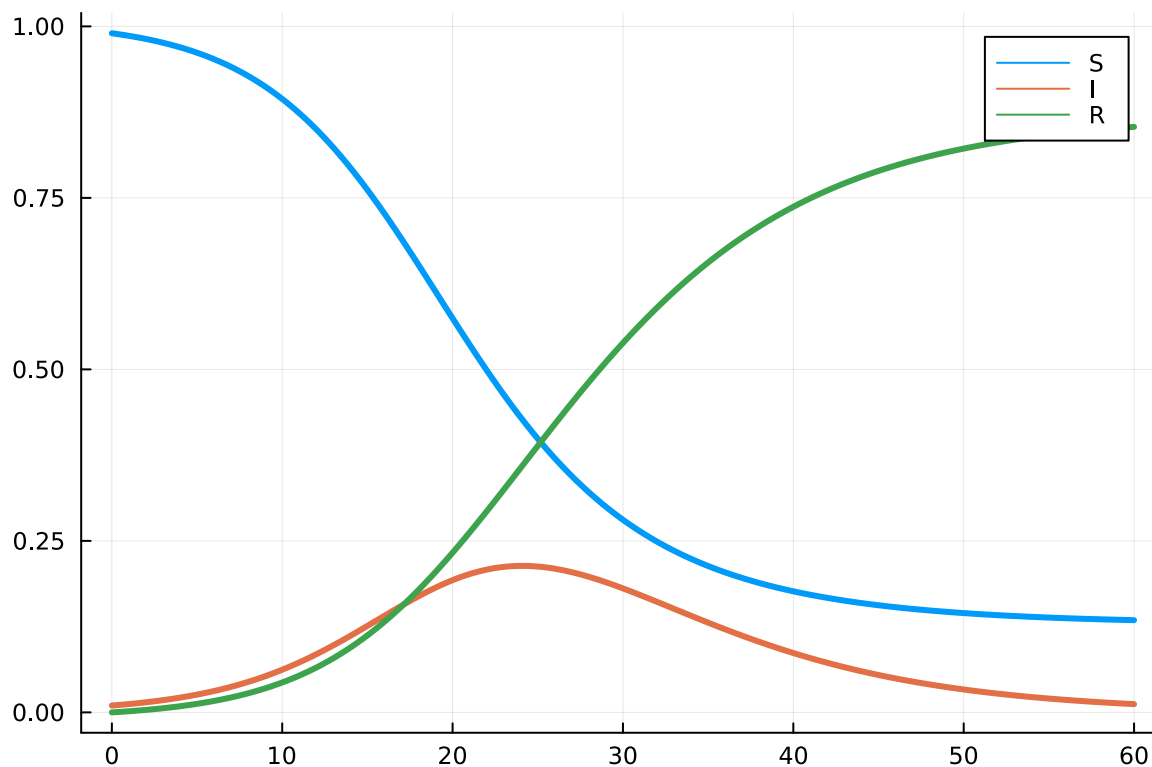
0.15

```
1 @bind γ Slider(0.0:0.01:1; show_value = true,default=0.15)
```

```
sir_results_new =
```

```
[[0.99, 0.01, 0.0], [0.989653, 0.0101965, 0.00015], [0.9893, 0.0103967, 0.000302947], [0.9889,
```

```
1 sir_results_new = euler_SIR(β, γ,  
2   [0.99, 0.01, 0.00],  
3   sir_T)
```



```
1 plot_sir!(plot(), sir_T, sir_results_new)
```

Exercise 3: Numerical gradient

For fitting we need optimization, and for optimization we will use *derivatives* (rates of change). In Exercise 1, we wrote a function `finite_difference_slope(f, a)` to approximate $f'(a)$. In this exercise we will write a function to compute *partial derivatives*.

Exercise 3.1

👉 Write functions $\partial_x(f, a, b)$ and $\partial_y(f, a, b)$ that calculate the **partial derivatives** $\frac{\partial f}{\partial x}$ and $\frac{\partial f}{\partial y}$ at (a, b) of a function $f : \mathbb{R}^2 \rightarrow \mathbb{R}$ (i.e. a function that takes two real numbers and returns one real).

Recall that $\frac{\partial f}{\partial x}$ is the derivative of the single-variable function $g(x) := f(x, b)$ obtained by fixing the value of y to b .

You should use **anonymous functions** for this. These have the form $x \rightarrow x^2$, meaning "the function that sends x to x^2 ".

∂x (generic function with 1 method)

```
1 function  $\partial x$ (f::Function, a, b)
2     h=10^-3
3     return (f(a+h,b)-f(a,b))/h
4 end
```

42.006999999999813

```
1  $\partial x$ (
2     (x, y) -> 7x^2 + y,
3     3, 7
4 )
```

Got it!

Yay ♥

∂y (generic function with 1 method)

```
1 function  $\partial y$ (f::Function, a, b)
2     h=10^-3
3     return (f(a,b+h)-f(a,b))/h
4 end
```

1.00000000000047748

```
1  $\partial y$ (
2     (x, y) -> 7x^2 + y,
3     3, 7
4 )
```

Got it!

Great! 🎉

Exercise 3.2

✎ Write a function `gradient(f, a, b)` that calculates the **gradient** of a function f at the point (a, b) , given by the vector $\nabla f(a, b) := (\frac{\partial f}{\partial x}(a, b), \frac{\partial f}{\partial y}(a, b))$.

`gradient` (generic function with 1 method)

```
1 function gradient(f::Function, a, b)
2
3     return [ $\partial x$ (f,a,b), $\partial y$ (f,a,b)]
4 end
```

[42.007, 1.0]

```
1 gradient(  
2   (x, y) -> 7x^2 + y,  
3   3, 7  
4 )
```

Got it!

Let's move on to the next section.

Exercise 4: Minimisation using gradient descent

In this exercise we will use **gradient descent** to find local **minima** of (smooth enough) functions, revisiting what we saw on the lectures.

As a refresher, we'll want to think of a function as a hill. To find a minimum we should "roll down the hill".

Exercise 4.1

We want to minimize a 1D function, i.e. a function $f : \mathbb{R} \rightarrow \mathbb{R}$. To do so we notice that the derivative tells us the direction in which the function *increases*. Positive slope means that the minimum is to the left, negative slope means to the right. So our *gradient descent method* is to take steps in the *opposite* direction, of a small size $\eta \cdot f'(x_0)$.

👉 Write a function `gradient_descent_1d_step(f, x0)` that performs a single gradient descent step, from the point `x0` and using your function `finite_difference_slope` to approximate the derivative. The result should be the next guess for x .

`gradient_descent_1d_step` (generic function with 1 method)

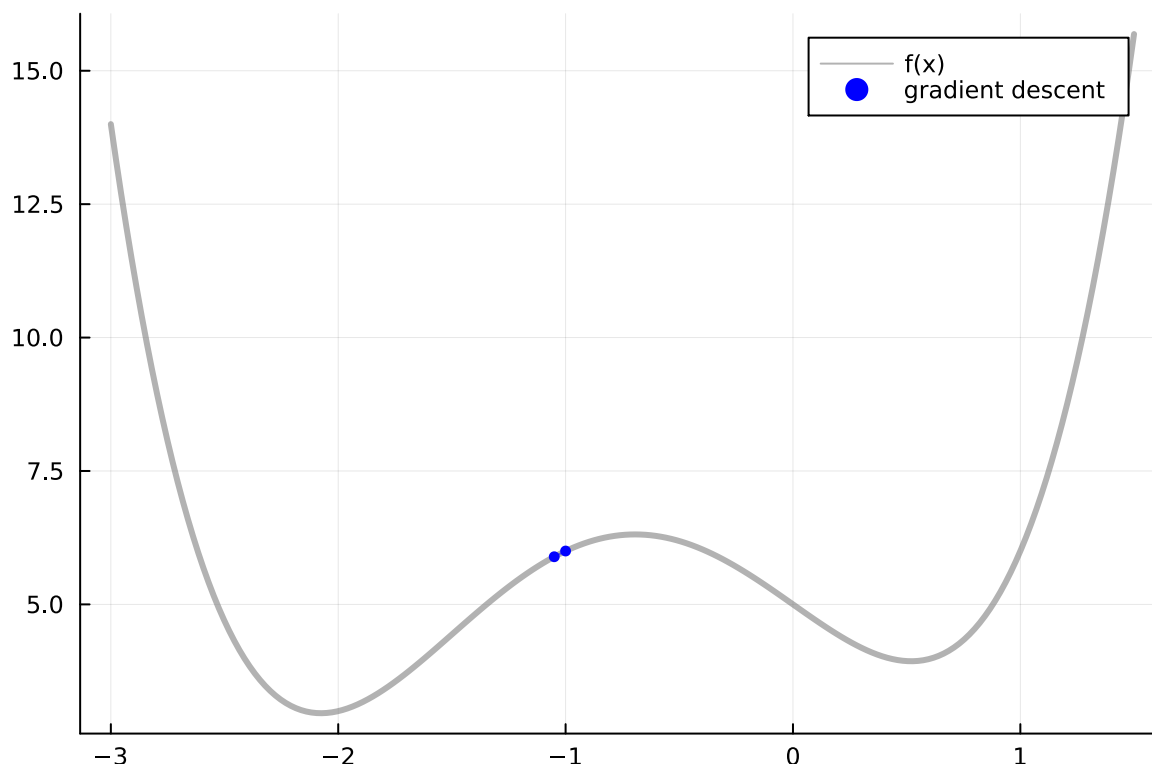
```
1 function gradient_descent_1d_step(f, x0; η=0.01)  
2  
3   return x0 - finite_difference_slope(f, x0) * η  
4 end
```

4.899989999999974

```
1 let  
2   f = x -> x^2  
3   # the minimum is at 0, so we should take a small step to the left  
4  
5   gradient_descent_1d_step(f, 5)  
6 end
```

Got it!

Yay ♥



$x_0 =$  -1.0

gradient_1d_viz (generic function with 1 method)

👉 Write a function `gradient_descent_1d(f, x0)` that repeatedly applies the previous function (`N_steps` times), starting from the point `x0`, like in the visualisation above. The result should be the final guess for x .

gradient_descent_1d (generic function with 1 method)

```
1 function gradient_descent_1d(f, x0; η=0.01, N_steps=1000)
2     for i in 1:N_steps
3         x0=gradient_descent_1d_step(f,x0,η=η)
4     end
5     return x0
6 end
```

4.9994999991586009

```
1 let
2     f = x -> (x - 5)^2 - 3
3     # minimum should be at x = 5
4     gradient_descent_1d(f, 0.0)
5 end
```

Got it!

Yay ♥

Right now we take a fixed number of steps, even if the minimum is found quickly. What would be a better way to decide when to end the function?

better_stopping_idea =

your answer here

```
1 better_stopping_idea = md"""
2 _your answer here_
3 """
```

Exericse 4.2

Multivariable calculus tells us that the gradient $\nabla f(a, b)$ at a point (a, b) is the direction in which the function *increases* the fastest. So again we should take a small step in the *opposite* direction. Note that the gradient is a *vector* which tells us which direction to move in the plane (a, b) . We multiply this vector with the scalar η to control the step size.

👉 Write functions `gradient_descent_2d_step(f, x0, y0)` and `gradient_descent_2d(f, x0, y0)` that do the same for functions $f(x, y)$ of two variables.

`gradient_descent_2d_step` (generic function with 1 method)

```
1 function gradient_descent_2d_step(f, x0, y0; η=0.01)
2     return [x0-(gradient(f,x0,y0)[1]*η), y0-(gradient(f,x0,y0)[2]*η)]
3 end
```

`gradient_descent_2d` (generic function with 1 method)

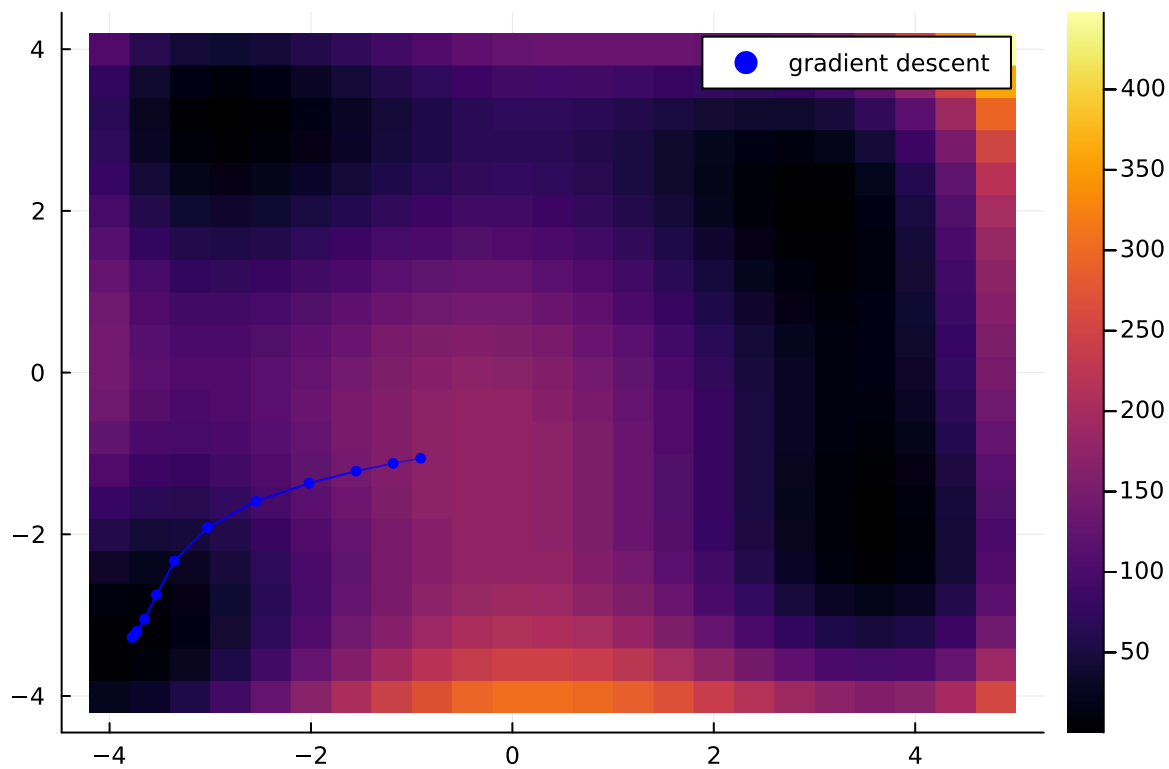
```
1 function gradient_descent_2d(f, x0, y0; η=0.01)
2     for i in 1:100000
3         x0,y0=gradient_descent_2d_step(f,x0,y0;η=η)
4     end
5     return [x0,y0]
6 end
```

[2.99957, 1.99976]

```
1 gradient_descent_2d(himmelbau, 0, 0)
```



```
1 @bind N_gradient_2d Slider(0:20)
```



$x_0 =$

$y_0 =$

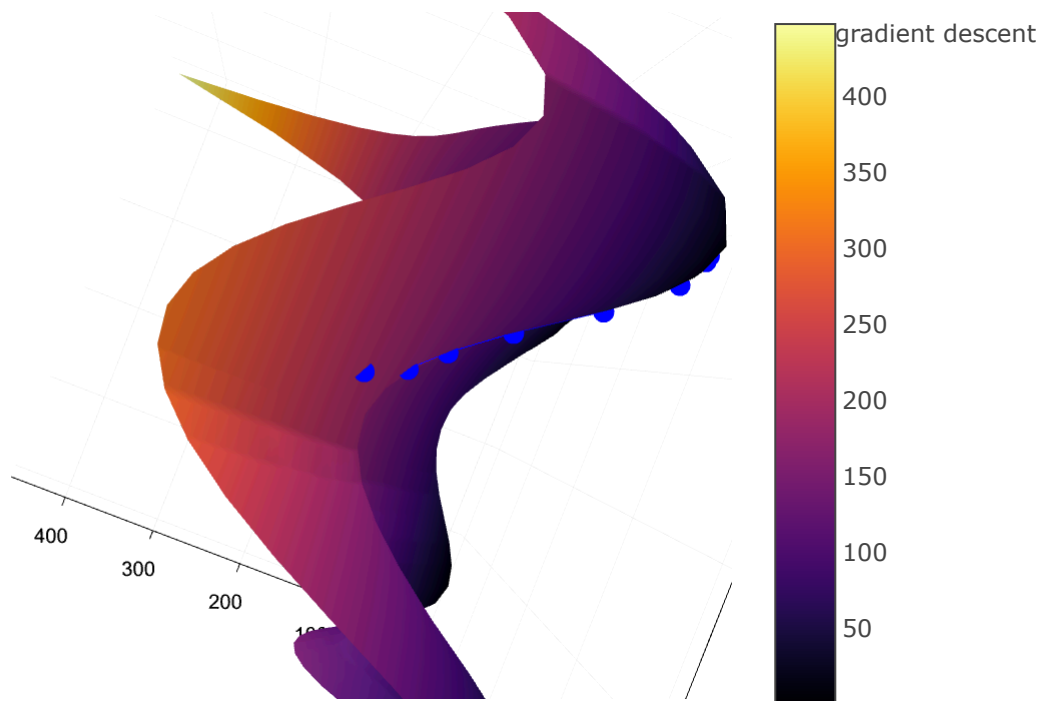
himmelbau (generic function with 1 method)

```
1 himmelbau(x, y) = (x^2 + y - 11)^2 + (x + y^2 - 7)^2
```

We also prepared a 3D visualisation if you like! It's a bit slow...

run_3d_visualisation = true

```
1 run_3d_visualisation = true
```

gradient_2d_viz_3d (generic function with 1 method)

gradient_2d_viz_2d (generic function with 1 method)

👉 Can you find different minima?

1 Enter cell code...

Exercise 5: Learning parameter values

In this exercise we will apply gradient descent to fit a simple function $y = f_{\alpha, \beta}(x)$ to some data given as pairs (x_i, y_i) . Here α and β are **parameters** that appear in the form of the function f . We want to find the parameters that provide the **best fit**, i.e. the version $f_{\alpha, \beta}$ of the function that is closest to the data when we vary α and β .

To do so we need to define what "best" means. We will define a measure of the distance between the function and the data, given by a **loss function**, which itself depends on the values of α and β . Then we will *minimize* the loss function over α and β to find those values that minimize this distance, and hence are "best" in this precise sense.

The iterative procedure by which we gradually adjust the parameter values to improve the loss function is often called **machine learning** or just **learning**, since the computer is "discovering" information in a gradual way, which is supposed to remind us of how humans learn. [Hint: This is not how humans learn.]

Exercise 5.1 - 🎲 frequencies

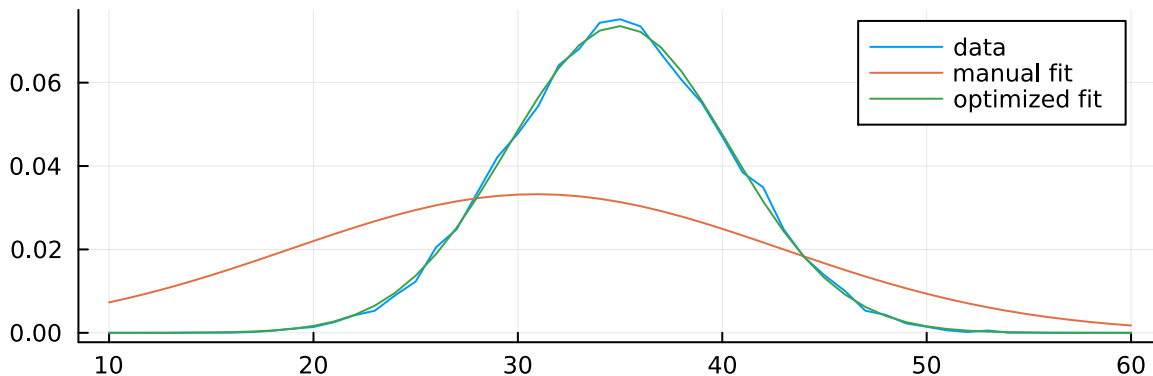
We generate a small dataset by throwing 10 dice, and counting the sum. We repeat this experiment many times, giving us a frequency distribution in a familiar shape.

```
1 import Statistics
```

dice_frequencies (generic function with 1 method)

```
(10:60, [0.0, 0.0, 0.0, 0.0, 5.0e-5, 5.0e-5, 5.0e-5, 0.0002, 0.0005, more ,0.0])
```

```
1 dice_x, dice_y = dice_frequencies(10, 20_000)
```



Let's try to fit a gaussian (normal) distribution. Its PDF with mean μ and standard deviation σ is

$$f_{\mu,\sigma}(x) := \frac{1}{\sigma\sqrt{2\pi}} \exp\left[-\frac{(x-\mu)^2}{2\sigma^2}\right]$$

👉 (Not graded) Manually fit a Gaussian distribution to our data by adjusting μ and σ until you find a good fit.

$\mu =$

$\sigma =$

Show manual fit: ☒

gauss (generic function with 1 method)

```
1 function gauss(x, μ, σ)
2     (1 / (sqrt(2π) * σ)) * exp(-(x-μ)^2 / σ^2 / 2)
3 end
```

What we just did was adjusting the function parameters until we found the best possible fit. Let's automate this process! To do so, we need to quantify how *good or bad* a fit is.

👉 Define a **loss function** to measure the "distance" between the actual data and the function. It will depend on the values of μ and σ that you choose:

$$\mathcal{L}(\mu, \sigma) := \sum_i [f_{\mu,\sigma}(x_i) - y_i]^2$$

loss_dice (generic function with 1 method)

```
1 function loss_dice( $\mu$ ,  $\sigma$ )
2   return sum((gauss.(dice_x, $\mu$ , $\sigma$ )-dice_y).^2)
3 end
```

false

```
1 loss_dice(guess_ $\mu$  + 3, guess_ $\sigma$ ) >
2 loss_dice(guess_ $\mu$ , guess_ $\sigma$ )
```

👉 Use your `gradient_descent_2d` function to find a local minimum of $\mathcal{L}$, starting with initial values $\mu = 30$ and $\sigma = 1$. Call the found parameters `found_ $\mu$` and `found_ $\sigma$` .

(34.9399, 5.42304)

```
1 found_ $\mu$ , found_ $\sigma$  = let
2
3   # your code here
4   x,y=gradient_descent_2d(loss_dice,30,1; $\eta$ =0.1)
5
6   x, y
7 end
```

Go back to the graph to see your optimized gaussian curve!

If your fit is close, then probability theory tells us that the found parameter μ should be close to the *weighted mean* of our data, and σ should approximate the *sample standard deviation*. We have already computed these values, and we check how close they are:

0.0307332537776972

```
1 abs(stats_ $\mu$  - found_ $\mu$ )
```

0.04549616588778971

```
1 abs(stats_ $\sigma$  - found_ $\sigma$ )
```

Got it!

Great!

stats_ μ = 34.9706

stats_ σ = 5.3775399245379845

Exercise 6: Putting it all together — fitting an SIR model to data

In this exercise we will fit the (non-spatial) SIR ODE model from Exercise 2 to some data generated from a different, *spatial* epidemic model, as we mentioned at the beginning. If we are able to find a good fit, that would suggest that the spatial aspect "does not matter" too much for the dynamics of

these models. If the fit is not so good, perhaps there is an important effect of space. (As usual in statistics, and indeed in modelling in general, we should be very cautious of making claims of this nature.)

This fitting procedure will be different from that in Exercise 4, however: we no longer have an explicit form for the function that we are fitting. Rather, it is simply the output of an ODE! So what should we do?

We will try to find the parameters β and γ for which *the output of the ODEs when we simulate it with those parameters* best matches the data!

Exercise 6.1

Below is the result from the spatial model. These are the *average S, I, R fractions* from running 20 simulations. Click on it!

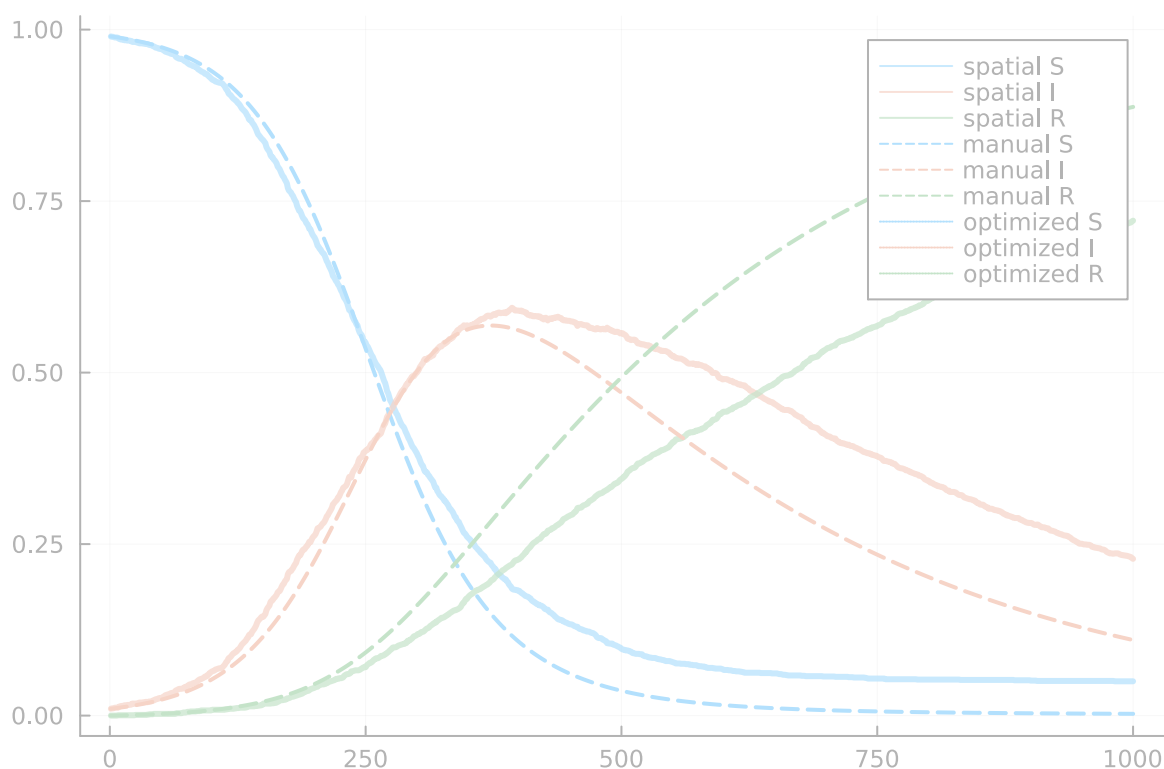
```
spatial_T = 1:1000
```

```
1 spatial_T = 1:length(spatial_results)
```

```
spatial_results =
```

```
[[0.99, 0.01, 0.0], [0.9895, 0.0105, 0.0], [0.9895, 0.0105, 0.0], [0.989, 0.011, 0.0], [0.988,
```

```
1 spatial_results = collect.(zip(hw4_results_transposed...))
```



👉 (Not graded) Manually fit the SIR curves to our data by adjusting β and γ until you find a good fit. This will use the `euler_SIR` function from Exercise 2.

$\beta =$  0.0207

$\gamma =$  0.0031

Show manual fit: ☒

👉 To do this automatically, we will again need to define a loss function $\mathcal{L}(\beta, \gamma)$. This will compare *the solution of the SIR equations* with parameters β and γ with your data.

This time, instead of comparing two vectors of numbers, we need to compare two vectors of *vectors*, the S, I, R values.

loss_sir (generic function with 1 method)

```
1 function loss_sir( $\beta$ ,  $\gamma$ )
2   sir_results_new2 = euler_SIR( $\beta$ ,  $\gamma$ , [0.99, 0.01, 0.0], 1:length(spatial_results))
3
4   total_loss = 0.0
5   for i in 1:length(sir_results_new2)
6     squared_diff = (sir_results_new2[i] .- spatial_results[i]) .^ 2
7     total_loss += sum(squared_diff)
8   end
9
10  return total_loss
11 end
```

29.602130180807446

```
1 loss_sir(guess_ $\beta$ , guess_ $\gamma$ )
```

👉 Use this loss function to find the optimal parameters β and γ .

[NaN, NaN]

```
1 found_ $\beta$ , found_ $\gamma$  = let
2
3   # your code here
4   gradient_descent_2d(loss_sir, 0.208, 0.0026;  $\eta=10^{-5}$ )
5 end
```

Hint

The found parameters are NaN, which means that finding good values for β and γ is not possible. Try setting a small value for η (e.g. 10^{-5}).

Function library

Just some helper functions used in the notebook.

hint (generic function with 1 method)

almost (generic function with 1 method)

still_missing (generic function with 2 methods)

keep_working (generic function with 2 methods)

yays =

[Fantastic!, Splendid!, Great!, Yay ♥, Great! 🎉, Well done!, Keep it up!, Good job!, Awesome!,



correct (generic function with 2 methods)

not_defined (generic function with 1 method)

TODO = **TODO**